

Lecture 2: Communication (Remote Procedure Calls – RPC)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2026

Today's Topic: Communication

Implementing Remote Procedure Calls

Andrew D. Birrell and Bruce Jay Nelson, ACM Trans. Comput. Syst., 2(1):39-59, 1984.

SIGOPS Hall of Fame Award

Optional:

A Critique of the Remote Procedure Call

Andrew S. Tanenbaum Robbert van Renesse, 1988

Why Today's Paper?

- Communication essential for distributed systems
- Popular, fundamental paradigm for communication
- RPC still used, many different implementations, project
- Detailed design and implementation choices
 - How do to binding/naming
 - Handling errors: Server and client crashes; lost and duplicate messages
 - Semantics (at-most-once vs. at-least-once)

Goals

Make distributed computation easy

- Non-experts
- As easy as local procedure calls

Highly efficient

- Within 5x network transmission time
- No need for programmers to avoid communication

Powerful semantics

- Won't need to add on to RPC
- At-most-once

Secure communication

- Encrypt text

Hide/handle Failures

Generality

- Can use for all communication

Communication Alternatives?

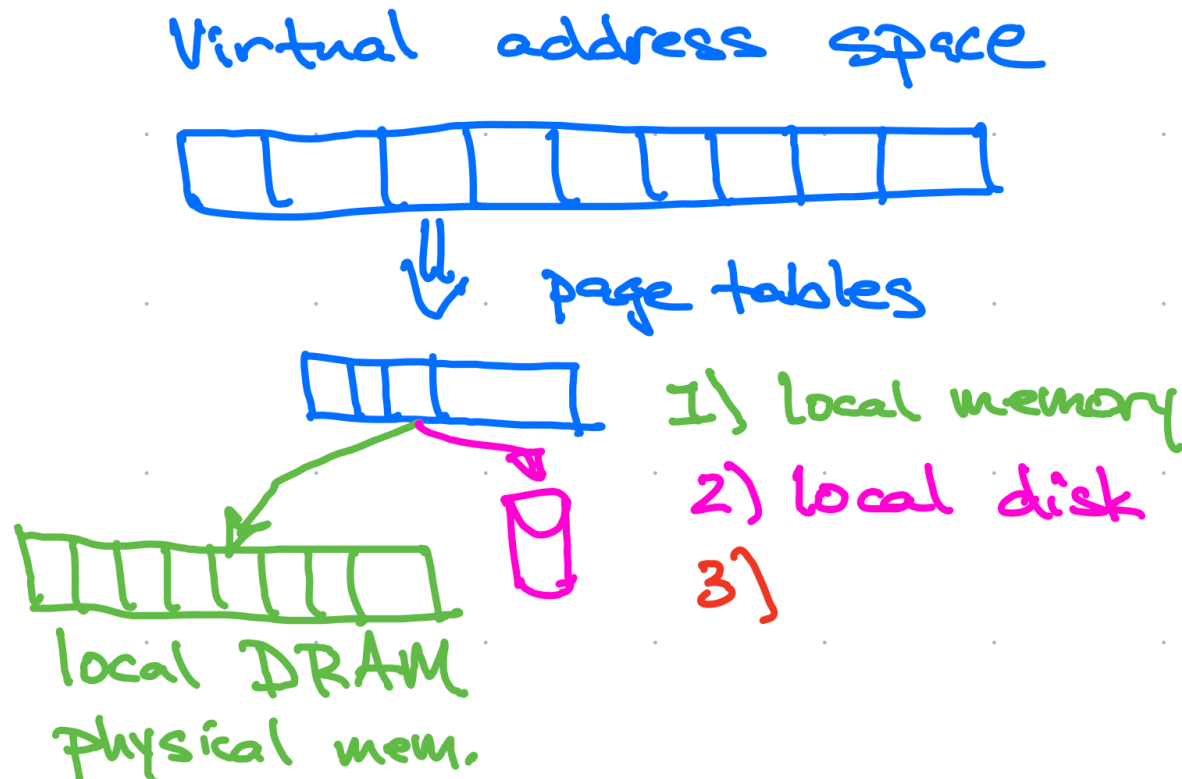
1) Low-level message passing (TCP/IP, UDP/IP)

Pros and cons?

- + Good performance
- + Can do anything
- Not easy to use, does not hide failures

2) Distributed Shared Memory

Alternative: Distributed Shared Memory (DSM)



Problems?

Performance: Ok, but not predictable; Mismatch between access gran + xfer gran

General: No, can't distribute computation, just data
Synchronization?

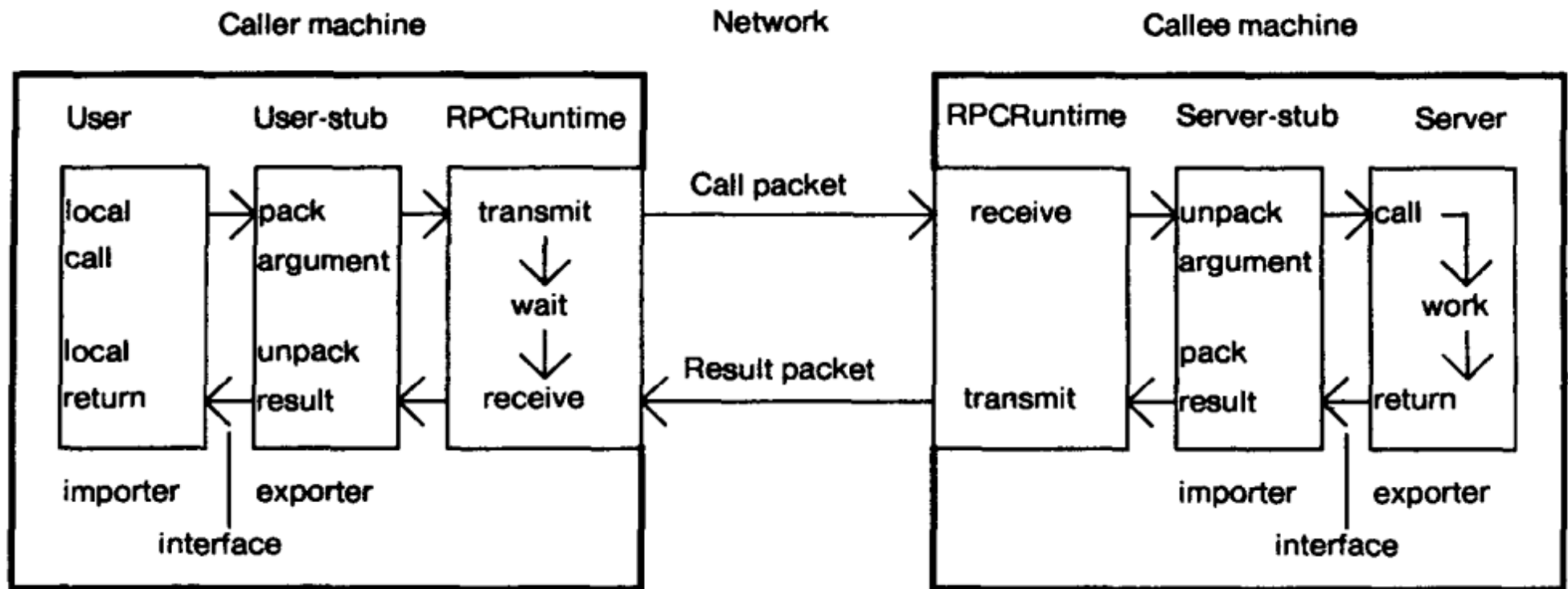
Ease: ok

Failures: No, doesn't hide

Communication Comparison

	Performance	Ease of Use	Generality	Handle Failures
Raw Messages	Good	Poor	Good	Prog must handle
DSM	Not predictable	Good	No, can't distribute computation	No
RPC	Good, but slower than local call Not good for bulk xfer Can't overlap communication (WAN)	Good	Medium - Pointers - Global variables - Comm back to client tricky	Ok

RPC Basics



What does stub library layer look like?

Client: `rc = foo(5);`

Client stub:

```
int foo(int arg) {  
    m = msg_create();  
    pack_int(m, arg);  
    send_msg(DEST, m);  
    m2 = wait_response();  
    rc = unpack_int(m2);  
    return rc;  
}
```

Advantages?

Automatically generate stubs
Save time, avoid errors

Generality Challenge:

How to pass/deference pointer args?

```
int bar (int *ptr)
```

What does original RPC specification do?

“programmer must avoid”

Can't pass, no shared address space, can't deref ptr on server

Other options?

Call by copy/restore

Caller: dereference, pass value to callee

Callee: If modified, pass back new values, copy back

Problems?

Synchronization, deep copies, performance cost

Binding: How to identify server?

Client c: Wants to use KVstore service with API get() and put()

Leverage Grapevine for naming: distributed replicated database

KVstore	a, b
a	1.2 (address)
b	1.3 (address)

Server d: export ("Kvstore", my-kv-dispatcher)

Grapevine: check d is valid Kvstore; add d

What is d's my-kv-dispatcher? Why?

{ Case 0: get(); break; case 1: set(); break; }
protect which functions can be called

What state does Server d maintain?

Export Table

Service	Dispatcher	Unique Service Id
Lock-service	My-lock-dispatch	2837519
Kvstore	My-kv-dispatch	3456789

How to set unique service id? Based on clock at start-up

Why is unique service id important? Won't match if crash

Client c: Import("Kvstore")

c->grapevine: grapevine -> d, 1.5 address

c->d ("Kvstore"): returns <tbl-index, unique service id>

c->d ("get"): "call packet" sends... <tbl-index, unique-service-id, get-id>

Transport Layer

Goals?

Performance – fast response time

Low load on server – scale to many clients (NFS + AFS)

Not much state per client

No handshaking overhead

Assumptions?

No large messages, local area

Semantics of RPC calls

- 1) Local machine procedure call guarantee: Exactly once
- 2) At-Least-Once: May be multiple times, 1 or more (not RPC!)

When ok? If ops are idempotent – same result if executed multiple times

Examples? Anything that just reads, pwrite(offset)

Not idempotent: add, append (increase file size)

Why not okay for RPC? Arbitrary functions

- 3) **RPC: At-most-once – 0 or 1 times**
(1 time, or 0 or 1 with error notification)

At-most-once Semantics

General idea for providing?

Detect duplicate calls, discard duplicates, return previous result

How many responses must server remember?

Just 1, last 1

Implications?

Can't have multiple outstanding requests to server

At-most-Once Details: Acknowledgements

C->s: call packet {call-id, procedure, args}

S->c: {call-id, results}

How is procedure specified? <table index, unique service id, entry pt id>

How can client know server received call?

Naïve: Send ack back (w/ call-id)

Optimization: Wait until have results

How can server know client received result? (needed to discard old result)

Naïve: Make client send ack back (w/ call-id)

Optimization: Next request indicates

Assumption? Only 1 outstanding request

At-most-Once Details: Call-id

C->s: call packet {**call-id**, procedure, args}

S->c: {**call-id**, results}

call-id: {[calling machine id, process], sequence number}

Why call-id (sequence number) in result?

Know which result; not old lost retransmitted result

Why call-id (sequence number) in call packet?

Call Table on Server

Activity	Last Sequence #	Last Result
{machine id, PID}	L	R
1.5, 9999	8	"hello world"

Msg arrives with seq # I
i > L ? New, execute
i = L ? Old, resend result
i < L ? Old delayed msg,
discard

State per Connection?

Important to minimize state on servers in distributed systems

- Can limit scalability (memory limits) for # clients

Server: Export Table + Call Table

Service	Dispatcher	Unique Service Id	Activity	Last Sequence #	Last Result
Lock-service	My-lock-dispatch	2837519	{machine id, PID}	L	R

How much state per connection on server? Small, one entry in call table
Can discard entry when not in danger of receiving retransmitted packet

Client: Last seq # (per ?); sequential or monotonic?

Just needs to be monotonically increasing

Can be across entire machine, can be for all services

What happens on Failures?

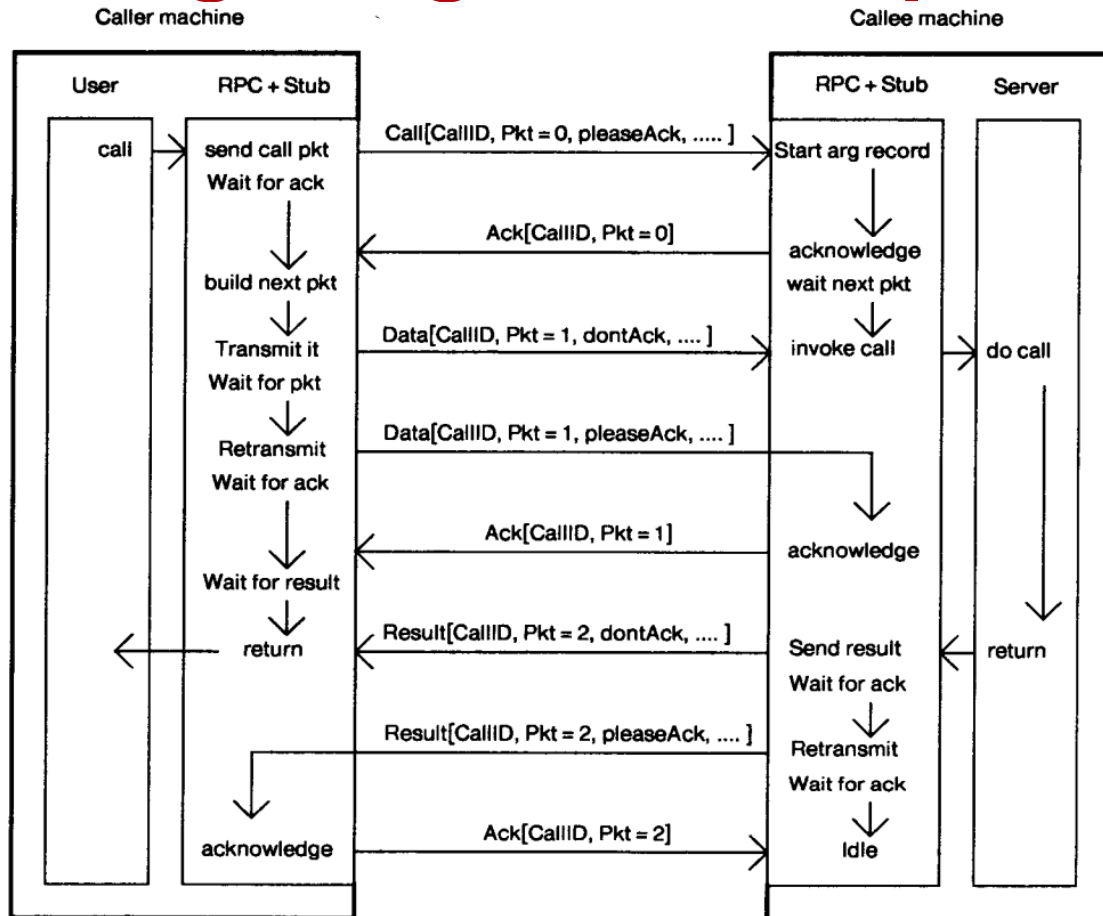
Server crash:

- Lose contents of Export Table, reboot, start up services again
- New Unique Service ID → Breaks binding with client
- Client informed on next interaction, or re-send prev call → Notified of crash
- **If re-send, what happened to last RPC before crash?**
 - Maybe exec, maybe not → Notify client of crash
 - “At-most once” (not exactly once)

Client crash:

- **What happens to ongoing RPCs?** Server continues, result wasted
- Reboot, **What is next sequence number?** Based on time, > old value (mono)
- **How does server respond?** Handles, replies normally as desired

Complicated Calls: Large Args w/ Multiple Packets



Implications?

Not optimized for bulk data xfer

Not good for long msg xfers

Not good for WAN

Evaluation

Did they show they met their goals?

Ease-of-use

Highly efficient

Powerful semantics

Secure communication (turn off encryption)

Hide failures

Generality

Procedure	Minimum	Median	Transmission	Local-only
no args/results	1059	1097	131	9
1 arg/result	1070	1105	142	10
2 args/results	1077	1127	152	11
4 args/results	1115	1171	174	12
10 args/results	1222	1278	239	17
1 word array	1069	1111	131	10
4 word array	1106	1153	174	13
10 word array	1214	1250	239	16
40 word array	1643	1695	566	51
100 word array	2915	2926	1219	98
resume except'n	2555	2637	284	134

Conclusions

RPC still very popular programming paradigm

- Assumes client/server, not bulk data xfer, only 1 outstanding request/client
- Can be built on other messaging protocols (UDP or TCP)

Leverage Grapevine for naming: distributed replicated database

RPC: At-most-once semantics

- Easier if could assume operations are **idempotent (at-least-once)**

Minimal state on server

- Track last seq-id + result (per activity)

Unique service id useful

- Identify server restarted

Sequence numbers useful

- Match responses/requests
- Identify duplicate requests (save last result)

Announcements

Reading Group 1: Meet with some members of reading group by Sunday

- Submit short individual write-up in Canvas
- Discuss past or future papers, could go over posted questions
- Will meet with same group for Week 2

Next: Failures

- Jim Gray. [Why Do Computers Stop And What Can Be Done About It?](#)
HP Labs Technical Report TR-85.7 **SIGOPS Hall of Fame Award**
- Aishwarya Ganesan, Ramnatthan Alagappan,
Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau,
[Redundancy Does Not Imply Fault Tolerance: Analysis of Distributed Storage
Reactions to Single Errors and Corruptions,](#)
Proc. of 15th USENIX Conf. on File and Storage Technologies (FAST '17)